

附录A 复习题答案

A.1 第1章复习题答案

1.完美的可移植程序是，其源代码无需修改就能在不同计算机系统中成功编译的程序。

2.源代码文件包含程序员使用的任何编程语言编写的代码。目标代码文件包含机器语言代码，它不必是完整的程序代码。可执行文件包含组成可执行程序的完整机器语言代码。

3.（1）定义程序目标；（2）设计程序；（3）编写程序；（4）编译程序；（5）运行程序；（6）测试和调试程序；（7）维护和修改程序。

4.编译器把源代码（如，用C语言编写的代码）翻译成等价的机器语言代码（也叫作目标代码）。

5.链接器把编译器翻译好的源代码以及库代码和启动代码组合起来，生成一个可执行程序。

A.2 第2章复习题答案

1.它们都叫作函数。

2.语法错误违反了组成语句或程序的规则。这是一个有语法错误的英文例子：Me speak English good.。这是一个有语法错误的C语言例子：printf"Where are the parentheses?";。

3.语义错误是指含义错误。这是一个有语义错误的英文例子：This sentence is excellent Czech.[\[1\]](#)。这是一个有语义错误的C语言例子：thrice_n = 3 + n;[\[2\]](#)。

4.第1行：以一个#开始；studio.h应改成stdio.h；然后用一对尖括号把

stdio.h括起来。

第2行：把{}改成(); 注释末尾把/*改成*/。

第3行：把(改成 {

第4行：int s末尾加上一个分号。

第5行没问题。

第6行：把:=改成，赋值用=，而不是用:=（这说明Indiana Sloth了解Pascal）。另外，用于赋值的值56也不对，一年有52周，不是56周。

第7行应该是：printf("There are %d weeks in a year.\n", s);

第9行：原程序中没有第9行，应该在该行加上一个右花括号}。

修改后的程序如下：

```
#include <stdio.h>

int main(void) /* this prints the number of weeks in a year */

{

int s;

s = 52;

printf("There are %d weeks in a year.\n", s);

return 0;

}
```

5.a.Baa Baa Black Sheep.Have you any wool?（注意，Sheep.和Have之间没有空格）

b.Begone!

O creature of lard!

c.What?

No/nfish?

(注意斜杠/和反斜杠\的效果不同，/只是一个普通的字符，原样打印)

d. $2 + 2 = 4$

(注意，每个%d与列表中的值相对应。还要注意，+的意思是加法，可以在printf()语句内部计算)

6.关键字是int和char (main是一个函数名；function是函数的意思；=是一个运算符)。

7.printf("There were %d words and %d lines.\n", words, lines);

8.执行完第7行后，a是5，b是2。执行完第8行后，a和b都是5。执行完第9行后，a和b仍然是5 (注意，a不会是2，因为在执行a = b;时，b的值已经被改为5)。

9.执行完第7行后，x是10，b是5。执行完第8行后，x是10，y是15。执行完第9行后，x是150，y是15。

A.3 第3章复习题答案

1.a.int类型，也可以是short类型或unsigned short类型。人口数是一个整数。

b.float类型，价格通常不是一个整数 (也可以使用double类型，但实际上不需要那么高的精度)。

c.char类型。

d.int类型，也可以是unsigned类型。

2.原因之一：在系统中要表示的数超过了int可表示的范围，这时要使用long类型。原因之二：如果要处理更大的值，那么使用一种在所有系统上都保证至少是 32 位的类型，可提高程序的可移植性。

3.如果要正好获得32位的整数，可以使用int32_t类型。要获得可储存至少32位整数的最小类型，可以使用int_least32_t类型。如果要为32位整数提供最快的计算速度，可以选择int_fast32_t类型（假设你的系统已定义了上述类型）。

4.a.char类型常量（但是储存为int类型）

b.int类型常量

c.double类型常量

d.unsigned int类型常量，十六进制格式

e.double类型常量

5.第1行：应该是#include <stdio.h>

第2行：应该是int main(void)

第3行：把(改为 {

第4行：g和h之间的;改成,

第5行：没问题

第6行：没问题

第7行：虽然这数字比较大，但在e前面应至少有一个数字，如1e21或1.0e21都可以。

第8行：没问题，至少没有语法问题。

第9行：把)改成}

除此之外，还缺少一些内容。首先，没有给rate变量赋值；其次未使用h变量；而且程序不会报告计算结果。虽然这些错误不会影响程序的运行（编译器可能给出变量未被使用的警告），但是它们确实与程序设计的初衷不符合。另外，在该程序的末尾应该有一个return语句。

下面是一个正确的版本，仅供参考：

```
#include <stdio.h>

int main(void)

{

float g, h;

float tax, rate;

rate = 0.08;

g = 1.0e5;

tax = rate*g;

h = g + tax;

printf("You owe $%f plus $%f in taxes for a total of $%f.\n", g, tax, h);

return 0;

}
```

6.

常量	类型	转换说明 (%转换字符)
12	int	%d
0X3	unsigned int	%#X
'C'	char (实际上是 int)	%c
2.34E07	double	%e
'\040'	char (实际上是 int)	%c
7.0	double	%f
6L	long	%ld
6.0f	float	%f
0x5.b6p12	float	%a

7.

常量	类型	转换说明 (%转换字符)
012	unsigned int	%#o
2.9e05L	long double	%Le
's'	char (实际上是 int)	%c
100000	long	%ld
'\n'	char (实际上是 int)	%c
20.0f	float	%f
0x44	unsigned int	%x
-40	int	%d

8.printf("The odds against the %d were %ld to 1.\n", imate, shot);printf("A score of %f is not an %c grade.\n", log, grade);

9.ch = '\r';

ch = 13;

ch = '\015'

ch = '\xd'

10.最前面缺少一行（第0行）： #include <stdio.h>

第1行： 使用/*和*/把注释括起来，或者在注释前面使用//。

第3行： int cows, legs;

第4行： country? \n");

第5行：把%c改为%d，把legs改为&legs。

第7行：把%f改为%d。

另外，在程序末尾还要加上return语句。

下面是修改后的版本：

```
#include <stdio.h>

int main(void) /* this program is perfect */

{

    int cows, legs;

    printf("How many cow legs did you count?\n");

    scanf("%d", &legs);

    cows = legs / 4;

    printf("That implies there are %d cows.\n", cows);

    return 0;

}
```

11.a.换行字符

b.反斜杠字符

c.双引号字符

d.制表字符

A.4 第4章复习题答案

1.程序不能正常运行。第1 个scanf()语句只读取用户输入的名，而用户输入的姓仍留在输入缓冲区中（缓冲区是用于储存输入的临时存储区）。下一条scanf()语句在输入缓冲区查找重量时，从上次读入结束的地方开始读取。这样就把留在缓冲区的姓作为体重来读取，导致 scanf()读取失败。另一方面，如果在要求输入姓名时输入Lasha 144，那么程序会把144作为用户的体重（虽然用户是在程序提示输入体重之前输入了144）。

2.a.He sold the painting for \$234.50.

b.Hi!（注意，第1个字符是字符常量；第2个字符由十进制整数转换而来；第3个字符是八进制字符常量的ASCII表示）

c.His Hamlet was funny without being vulgar.has 42 characters.

d.Is 1.20e+003 the same as 1201.00?

3.在这条语句中使用\\": printf("\\\"%s\\\"\\nhas %d characters.\\n", Q, strlen(Q));

4.下面是修改后的程序：

```
#include <stdio.h>      /* 别忘了要包含合适的头文件 */
```

```
#define B "booboo"      /* 添加#、双引号 */
```

```
#define X 10             /* 添加# */
```

```
int main(void)           /* 不是main(int) */
```

```
{
```

```
int age;
```

```
int xp;                  /* 声明所有的变量 */
```

```
char name[40];           /* 把name声明为数组 */
```



```

printf("Please enter your first name.\n"); /* 添加\n, 提高可读性 */

scanf("%s", name);

printf("All right, %s, what's your age?\n", name); /* %s用于打印字符串*/

scanf("%d", &age); /* 把%f改成%d, 把age改成&age */

xp = age + X;

printf("That's a %s! You must be at least %d.\n", B, xp);

return 0; /* 不是rerun */

}

```

5.记住, 要打印%必须用%%:

```

printf("This copy of \"%s\" sells for $%0.2f.\n", BOOK, cost);

printf("That is %0.0f%% of list.\n", percent);

```

6.a.%d

b.%4X

c.%10.3f

d.%12.2e

e.%-30s

7.a.%15lu

b.%#4x

c.%-12.2E

d.%+10.3f

e.%8.8s

8.a.%6.4d

b.%*o

c.%2c

d.%+0.2f

e.%-7.5s

9.a.int dalmations;

scanf("%d", &dalmations);

b.float kgs, share;

scanf("%f%f", &kgs, &share);

（注意：对于本题的输入，可以使用转换字符e、f和g。另外，除了%c之外，在%和转换字符之间加空格不会影响最终的结果）

c.char pasta[20];

scanf("%s", pasta);

d.char action[20];

int value;

scanf("%s %d", action, &value);

e.int value;

```
scanf("%*s %d", &value);
```

10.空白包括空格、制表符和换行符。C 语言使用空白分隔记号。scanf() 使用空白分隔连续的输入项。

11.%z 中的 z 是修饰符，不是转换字符，所以要在修饰符后面加上一个它修饰的转换字符。可以使用%zd打印十进制数，或用不同的说明符打印不同进制的数，例如，%zx打印十六进制的数。

12.可以分别把(和)替换成{和}。但是预处理器无法区分哪些圆括号应替换成花括号，哪些圆括号不能替换成花括号。因此，

```
#define ( {  
  
#define ) }  
  
int main(void)  
  
(  
  
printf("Hello, O Great One!\n");  
  
)
```

将变成：

```
int main{void}  
  
{  
  
printf{"Hello, O Great One!\n"};  
  
}
```

A.5 第5章复习题答案

1.a.30

b.27（不是3）。 $(12+6)/(2*3)$ 得3。

c. $x = 1$ ， $y = 1$ （整数除法）。

d. $x = 3$ （整数除法）， $y = 9$ 。

2.a.6（由 $3 + 3.3$ 截断而来）

b.52

c.0（ $0 * 22.0$ 的结果）

d.13（ $66.0 / 5$ 或13.2，然后把结果赋给int类型变量）

3.a.37.5（ $7.5 * 5.0$ 的结果）

b.1.5（ $30.0 / 20.0$ 的结果）

c.35（ $7 * 5$ 的结果）

d.37（ $150 / 4$ 的结果）

e.37.5（ $7.5 * 5$ 的结果）

f.35.0（ $7 * 5.0$ 的结果）

4.第0行：应增加一行`#include <stdio.h>`。

第3行：末尾用分号，而不是逗号。

第6行：`while`语句创建了一个无限循环。因为`i`的值始终为1，所以它总是小于30。推测一下，应该是想写`while(i++ < 30)`。

第6～8行：这样的缩进布局不能使第7行和第8行组成一个代码块。由于没有用花括号括起来，`while`循环只包括第7行，所以要添加花括号。

第7行：因为1和i都是整数，所以当i为1时，除法的结果是1；当i为更大的数时，除法结果为0。用`n = 1.0/i`，i在除法运算之前会被转换为浮点数，这样就能得到非零值。

第8行：在格式化字符串中没有换行符（`\n`），这导致数字被打印成一行。

第10行：应该是`return 0;`

下面是正确的版本：

```
#include <stdio.h>

int main(void)

{

int i = 1;

float n;

printf("Watch out! Here come a bunch of fractions!\n");

while (i++ < 30)

{

n = 1.0/i;

printf(" %f\n", n);

}

printf("That's all, folks!\n");

return 0;
```

```
}
```

5.这个版本最大的问题是测试条件（sec是否大于0？）和scanf()语句获取sec变量的值之间的关系。具体地说，第一次测试时，程序尚未获得sec的值，用来与0作比较的是正好在sec变量内存位置上的一个垃圾值。一个比较笨拙的方法是初始化 sec（如，初始化为 1）。这样就可通过第一次测试。不过，还有另一个问题。当最后输入0结束程序时，在循环结束之前不会检查sec，所以0也被打印了出来。因此，更好的方法是在while测试之前使用scanf()语句。可以这样修改：

```
scanf("%d", &sec);

while ( sec > 0 ) {

min = sec/S_TO_M;

left = sec % S_TO_M;

printf("%d sec is %d min, %d sec.\n", sec, min, left);

printf("Next input?\n");

scanf("%d", &sec);

}
```

while循环第一轮迭代使用的是scanf()在循环外面获取的值。因此，在while循环的末尾还要使用一次scanf()语句。这是处理类似问题的常用方法。

6.下面是该程序的输出：

```
%s! C is cool!
```

```
! C is cool!
```

```
11
```

11

12

11

解释一下。第1个printf()语句与下面的语句相同：

```
printf("%s! C is cool!\n", "%s! C is cool!\n");
```

第2个printf()语句首先把num递增为11，然后打印该值。第3个printf()语句打印num的值（值为11）。第4个printf()语句打印n当前的值（仍为12），然后将其递减为11。最后一个printf()语句打印num的当前值（值为11）。

7.下面是该程序的输出：

```
SOS:4 4.00
```

表达式c1 - c2的值和'S' - '0'的值相同（其对应的ASCII值是83 - 79）。

8.把1~10打印在一行，每个数字占5列宽度，然后开始新的一行：

```
1 2 3 4 5 6 7 8 9 10
```

9.下面是一个参考程序，假定字母连续编码，与ASCII中的情况一样。

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
char ch = 'a';
```

```
while (ch <= 'g')
```

```
printf("%5c", ch++);
```

```
printf("\n");  
  
return 0;  
  
}
```

10.下面是每个部分的输出：

a.1 2

注意，先递增x的值再比较。光标仍留在同一行。

b.101

102

103

104

注意，这次x先比较后递增。在示例a和b中，x都是在先递增后打印。另外还要注意，虽然第2个printf()语句缩进了，但是这并不意味着它是while循环的一部分。因此，在while循环结束后，才会调用一次该printf()语句。

c.stuvw

该例中，在第1次调用printf()语句后才会递增ch。

11.这个程序有点问题。while循环没有用花括号把两个缩进的语句括起来，只有printf()是循环的一部分，所以该程序一直重复打印消息COMPUTER BYTES DOG，直到强行关闭程序为止。

12.a.x = x + 10;

b.x++; or ++x; or x = x + 1;

`c.c = 2 * (a + b);`

`d.c = a + 2* b;`

13  `a.x--; or --x; or x = x - 1;`

`b.m = n % k;`

`c.p = q / (b - a);`

`d.x = (a + b) / (c * d);`

A.6 第6章复习题答案

1.2, 7, 70, 64, 8, 2。

2.该循环的输出是：

36 18 9 4 2 1

如果value是double类型，即使value小于1，循环的测试条件仍然为真。循环将一直执行，直到浮点数下溢生成0为止。另外，value是double类型时，%3d转换说明也不正确。

3.a.x > 5

b scanf("%lf",&x) != 1

c.x == 5

4.a scanf("%d", &x) == 1

b.x != 5

c.x >= 20

5.第4行： 应该是list[10]。

第6行：逗号改为分号。i的范围应该是0~9，不是1~10。

第9行：逗号改为分号。>=改成<=，否则，当i等于1时，该循环将成为无限循环。

第10行：在第10行和第11行之间少了一个右花括号。该右花括号与第7行的左花括号配对，形成一个for循环块。然后在这个右花括号与最后一个右花括号之间，少了一行return 0;。

下面是一个正确的版本：

```
#include <stdio.h>

int main(void)

{
    /* 第3行 */

    int i, j, list(10);          /* 第4行 */

    for (i = 1, i <= 10, i++)    /* 第6行 */

    {
        /* 第7行 */

        list[i] = 2*i + 3;      /* 第8行 */

        for (j = 1, j >= i, j++) /* 第9行 */

        printf(" %d", list[j]); /* 第10行 */

        printf("\n");          /* 第11行 */

    }

    return 0;

}
```

6.下面是一种方法:

```
#include <stdio.h>

int main(void)

{

int col, row;

for (row = 1; row <= 4; row++)

{

for (col = 1; col <= 8; col++)

printf("$");

printf("\n");

}

return 0;

}
```

7.a.Hi! Hi! Hi! Bye! Bye! Bye! Bye!

b.ACGM (因为代码中把int类型值与char类型值相加, 编译器可能警告会损失有效数字)

8.a.Go west, youn

b.Hp!xftu-!zpvo

c.Go west, young

d.\$o west, youn

9.其输入如下:

31|32|33|30|31|32|33|

1

5

9

13

2 6

4 8

8 10

=====

=====

=====

=====

=====

10.a.mint

b.10个元素

c.double 类型的值

d.第ii行正确，`mint[2]`是double类型的值，`&mingt[2]`是它在内存中的位置。

11.因为第1个元素的索引是0，所以循环的范围应该是0~SIZE - 1，而不是1~SIZE。但是，如果只是这样更改会导致赋给第1个元素的值是0，不是2。所以，应重写这个循环：

```
for (index = 0; index < SIZE; index++)
```

```
by_twos[index] = 2 * (index + 1);
```

与此类似，第2个循环的范围也要更改。另外，应该在数组名后面使用数组索引：

```
for( index = 0; index < SIZE; index++)
```

```
printf("%d ", by_twos[index]);
```

错误的循环条件会成为程序的定时炸弹。程序可能开始运行良好，但是由于数据被放在错误的位置，可能在某一时刻导致程序不能正常工作。

12.该函数应声明为返回类型为long，并包含一个返回long类型值的return语句。

13.把num的类型强制转换成long类型，确保计算使用long类型而不是int类型。在int为16位的系统中，两个int类型值的乘积在返回之前会被截断为一个int类型的值，这可能会丢失数据。

```
long square(int num)
```

```
{
```

```
return ((long) num) * num;  
  
}
```

14.输出如下:

1: Hi!

k = 1

k is 1 in the loop

Now k is 3

k = 3

k is 3 in the loop

Now k is 5

k = 5

k is 5 in the loop

Now k is 7

k = 7

A.7 第7章复习题答案

1.b是true。

2.a.number >= 90 && number < 100

b.ch != 'q' && ch != 'k'

c.(number >= 1 && number <= 9) && number != 5

d.可以写成!(number >= 1 && number <= 9), 但是number < 1 || number > 9更好理解。

3.第5行: 应该是scanf("%d %d", &weight, &height);。不要忘记scanf()中要用&。另外, 这一行前面应该有提示用户输入的语句。

第9行: 测试条件中要表达的意思是(height < 72 && height > 64)。根据前面第7行中的测试条件, 能到第9行的height一定小于72, 所以, 只需要用表达式(height > 64)即可。但是, 第6行中已经包含了height > 64这个条件, 所以这里完全不必再判断, if else应改成else。

第11行: 条件冗余。第2个表达式(weight不小于或不等于300)和第1个表达式含义相同。只需用一个简单的表达式(weight > 300)即可。但是, 问题不止于此。第 11 行是一个错误的if, 这行的else if与第6行的if匹配。但是, 根据if的“最接近规则”, 该else if应该与第9行的else if匹配。因此, 在weight小于100且小于或等于64时到达第11行, 而此时weight不可能超过300。

第7行~第10行: 应该用花括号括起来。这样第11行就确定与第6行匹配。但是, 如果把第9行的else if替换成简单的else, 就不需要使用花括号。

第13行: 应简化成if (height > 48)。实际上, 完全可以省略这一行。因为第12行已经测试过该条件。

下面是修改后的版本:

```
#include <stdio.h>

int main(void)
{

    int weight, height; /* weight in lbs, height in inches */

    printf("Enter your weight in pounds and ");
```

```
printf("your height in inches.\n");

scanf("%d %d", &weight, &height);

if (weight < 100 && height > 64)

if (height >= 72)

printf("You are very tall for your weight.\n");

else

printf("You are tall for your weight.\n");

else if (weight > 300 && height < 48)

printf(" You are quite short for your weight.\n");

else

printf("Your weight is ideal.\n");

return 0;

}
```

4.a.1。5确实大于2，表达式为真，即是1。

b.0。3比2大，表达式为假，即是0。

c.1。如果第 1 个表达式为假，则第 2 个表达式为真，反之亦然。所以，只要一个表达式为真，整个表达式的结果即为真。

d.6。因为 $6 > 2$ 为真，所以 $(6 > 2)$ 的值为1。

e.10。因为测试条件为真。

f.0。如果 $x > y$ 为真，表达式的值就是 $y > x$ ，这种情况下它为假或0。如果 $x > y$ 为假，那么表达式的值就是 $x > y$ ，这种情况下为假。

5.该程序打印以下内容：

```
*#%*#%$#%*#%*#%$#%*#%*#%$#%*#%*#%
```

无论怎样缩排，每次循环都会打印#，因为缩排并不能让`putchar('#);`成为`if else`复合语句的一部分。

6.程序打印以下内容：

```
fat hat cat Oh no!
```

```
hat cat Oh no!
```

```
cat Oh no!
```

7.第5行～第7行的注释要以`*/`结尾，或者把注释开头的`/*`换成`//`。表达式`'a' <= ch <= 'z'`应替换成`ch >= 'a' && ch <= 'z'`。

或者，包含 `ctype.h` 并使用 `islower()`，这种方法更简单，而且可移植性更高。顺带一提，虽然从 C 的语法方面看，`'a' <= ch <= 'z'`是有效的表达式，但是它的含义不明。因为关系运算符从左往右结合，该表达式被解释成`('a' <= ch) <= 'z'`。圆括号中的表达式的值不是1就是0（真或假），然后判断该值是否大于或等于'z'的数值码。1和0都不满足测试条件，所以整个表达式恒为0（假）。在第2个测试表达式中，应该把`||`改成`&&`。另外，虽然`!(ch < 'A')`是有效的表达式，而且含义也正确，但是用`ch >= 'A'`更简单。这一行的'z'后面应该有两个圆括号。更简单的方法是使用`isupper()`。在`uc++;`前面应该加一行`else`。否则，每输入一个字符，`uc`都会递增1。另外，在`printf()`语句中的格式化字符串应该用双引号括起来。下面是修改后的版本：

```
#include <stdio.h>
```

```

#include <ctype.h>

int main(void)

{

char ch;

int lc = 0; /*统计小写字母*/

int uc = 0; /*统计大写字母*/

int oc = 0; /*统计其他字母*/

while ((ch = getchar()) != '#')

{

if (islower(ch))

lc++;

else if (isupper(ch))

uc++;

else

oc++;

}

printf("%d lowercase, %d uppercase, %d other", lc, uc, oc);

return 0;

}

```

8.该程序将不停重复打印下面一行：

You are 65.Here is your gold watch.

问题出在这一行： `if (age = 65)`

这行代码把age设置为65，使得每次迭代的测试条件都为真。

9.下面是根据给定输入的运行结果：

q

Step 1

Step 2

Step 3

c

Step 1

h

Step 1

Step 3

b

Step 1

Done

注意，b和#都可以结束循环。但是输入b会使得程序打印step 1，而输入#则不会。

10.下面是一种解决方案:

```
#include <stdio.h>

int main(void)

{

char ch;

while ((ch = getchar()) != '#')

{

if (ch != '\n')

{

printf("Step 1\n");

if (ch == 'b')

break;

else if (ch != 'c')

{

if (ch != 'h')

printf("Step 2\n");

printf("Step 3\n");

}

}

}
```

```

}

printf("Done\n");

return 0;

}

```

A.8 第8章复习题答案

1.表达式 `putchar(getchar())`使程序读取下一个输入字符并打印出来。`getchar()`的返回值是`putchar()`的参数。但`getchar(putchar())`是无效的表达式，因为`getchar()`不需要参数，而`putchar()`需要一个参数。

2.a.显示字符H。

b.如果系统使用ASCII，则发出一声警报。

c.把光标移至下一行的开始。

d.退后一格。

3.`count <essay >essayct`或者`count >essayct <essay`

4.都不是有效的命令。

5.EOF是由`getchar()`和`scanf()`返回的信号（一个特殊值），表明函数检测到文件结尾。

6.a.输出是：If you qu

注意，字符I与字符i不同。还要注意，没有打印i，因为循环在检测到i之后就退出了。

b.如果系统使用ASCII，输出是：HJacrthjacrt

while的第1轮迭代中，为ch读取的值是H。第1个putchar()语句使用的ch的值是H，打印完毕后，ch的值加1（现在是ch的值是I）。然后到第2个putchar()语句，因为是++ch，所以先递增ch（现在ch的值是J）再打印它的值。然后进入下一轮迭代，读取输入序列中的下一个字符（a），重复以上步骤。需要注意的是，两个递增运算符只在ch被赋值后影响它的值，不会让程序在输入序列中移动。

7.C的标准I/O库把不同的文件映射为统一的流来统一处理。

8.数值输入会跳过空格和换行符，但是字符输入不会。假设有下面的代码：

```
int score;

char grade;

printf("Enter the score.\n");

scanf("%s", %score);

printf("Enter the letter grade.\n");

grade = getchar();
```

如果输入分数98，然后按下Enter键把分数发送给程序，其实还发送了一个换行符。这个换行符会留在输入序列中，成为下一个读取的值（grade）。如果在字符输入之前输入了数字，就应该在处理字符输入之前添加删除换行符的代码。

A.9 第9章复习题答案

1.形式参数是定义在被调函数中的变量。实际参数是出现在函数调用中的值，该值被赋给形式参数。可以把实际参数视为在函数调用时初始化形式参数的值。

2.a.void donut(int n)

b.int gear(int t1, int t2)

c.int guess(void)

d.void stuff_it(double d, double *pd)

3.a.char n_to_char(int n)

b.int digits(double x, int n)

c.double * which(double * p1, double * p2)

d.int random(void)

4.

```
int sum(int a, int b)
```

```
{
```

```
    return a + b;
```

```
}
```

5.用double替换int即可：

```
double sum(double a, double b)
```

```
{
```

```
    return a + b;
```

```
}
```

6.该函数要使用指针：

```

void alter(int * pa, int * pb)

{

int temp;

temp = *pa + *pb;

*pb = *pa - *pb;

*pa = temp;

}

```

或者：

```

void alter(int * pa, int * pb)

{

*pa += *pb;

*pb = *pa - 2 * *pb;

}

```

7.不正确。`num`应声明在`salami()`函数的参数列表中，而不是声明在函数体中。另外，把`count++`改成`num++`。

8.下面是一种方案：

```

int largest(int a, int b, int c)

{

int max = a;

```



```
if (b > max)
```

```
max = b;
```

```
if (c > max)
```

```
max = c;
```

```
return max;
```

```
}
```

9.下面是一个最小的程序，showmenu()和getchoice()函数分别是a和b的答案。

```
#include <stdio.h>
```

```
/* 声明程序中要用到的函数 */
```

```
void showmenu(void);
```

```
int getchoice(int, int);
```

```
int main()
```

```
{
```

```
int res;
```

```
showmenu();
```

```
while ((res = getchoice(1, 4)) != 4)
```

```
{
```

```
printf("I like choice %d.\n", res);
```

```

showmenu();

}

printf("Bye!\n");

return 0;

}

void showmenu(void)

{

printf("Please choose one of the following:\n");

printf("1) copy files          2) move files\n");

printf("3) remove files        4) quit\n");

printf("Enter the number of your choice:\n");

}

int getchoice(int low, int high)

{

int ans;

int good;

good = scanf("%d", &ans);

while (good == 1 && (ans < low || ans > high))

{

```

```

printf("%d is not a valid choice; try again\n", ans);

showmenu();

scanf("%d", &ans);

}

if(good != 1)

{

printf("Non-numeric input.");

ans = 4;

}

return ans;

}

```

A.10 第10章复习题答案

1.打印的内容如下：

8 8

4 4

0 0

2 2

2.数组ref有4个元素，因为初始化列表中的值是4个。

3.数组名ref指向该数组的首元素（整数8）。表达式ref + 1指向该数组的

第2个元素（整数4）。++ref不是有效的表达式，因为ref是一个常量，不是变量。

4.ptr指向第1个元素，ptr + 2指向第3个元素（即第2行的第1个元素）。

a.12和16。

b.12和14（初始化列表中，用花括号把12括起来，把14和16括起来，所以12初始化第1行的第1个元素，而14初始化第2行的第1个元素）。

5.ptr指向第1行，ptr + 1指向第2行。*ptr指向第1行的第1个元素，而*(ptr + 1)指向第2行的第1个元素。

a.12和16。

b.12和14（同第4题，12初始化第1行的第1个元素，而14初始化第2行的第1个元素）。

6.a.&grid[22][56]

b.&grid[22][0]或grid[22]

（grid[22]是一个内含100个元素的一维数组，因此它就是首元素grid[22][0]的地址。）

c.&grid[0][0]或grid[0]或(int *) grid

（grid[0]是int类型元素grid[0][0]的地址，grid是内含100个元素的grid[0]数组的地址。）

这两个地址的数值相同，但是类型不同，可以用强制类型转换把它们转换成相同的类型。）

7.a.int digits[10];

b.float rates[6];

c.int mat[3][5];

d.char * psa[20];

注意，[]比*的优先级高，所以在没有圆括号的情况下，psa先与[20]结合，然后再与*结合。因此该声明与char *(psa[20]);相同。

e.char (*pstr)[20];

注意

对第e小题而言，char *pstr[20];不正确。这会让pstr成为一个指针数组，而不是一个指向数组的指针。具体地说，如果使用该声明，pstr就指向一个char类型的值（即数组的第1个成员），而pstr + 1则指向下一个字节。使用正确的声明，pstr是一个变量，而不是一个数组名。而且pstr+ 1指向起始字节后面的第20个字节。

8.a.int sextet[6] = {1, 2, 4, 8, 16, 32};

b.sextet[2]

c.int lots[100] = { [99] = -1};

d.int pots[100] = { [5] = 101, [10] = 101, 101, 101, 101};

9.0~9

10.a.rootbeer[2] = value;有效。

b.scanf("%f", &rootbeer);无效，rootbeer不是float类型。

c.rootbeer = value;无效，rootbeer不是float类型。

d.printf("%f", rootbeer);无效，rootbeer不是float类型。

e.things[4][4] = rootbeer[3];有效。

f.things[5] = rootbeer;无效，不能用数组赋值。

g.pf = value;无效，value不是地址。

h.pf = rootbeer;有效。

11.int screen[800][600] ;

12.a.

void process(double ar[], int n);

void processvla(int n, double ar[n]);

process(trots, 20);

processvla(20, trots);

b.

void process2(short ar2[30], int n);

void process2vla(int n, int m, short ar2[n][m]);

process2(clops, 10);

process2vla(10, 30, clops);

c.

void process3(long ar3[10][15], int n);

void process3vla(int n, int m,int k, long ar3[n][m][k]);

process3(shots, 5);

process3vla(5, 10, 15, shots);

13.a.

```
show( (int [4]) {8,3,9,2}, 4);
```

b.

```
show2( (int[][3]){{8,3,9}, {5,4,1}}, 2);
```

A.11 第11章复习题答案

1.如果希望得到一个字符串，初始化列表中应包含'\0'。当然，也可以用另一种语法自动添加空字符：

```
char name[] = "Fess";
```

2.

See you at the snack bar.

ee you at the snack bar.

See you

e you

3.

y

my

mmy

ummy

Yummy

4.I read part of it all the way through.

5.a.Ho Ho Ho!!oH oH oH

b.指向char的指针（即，char *）。

c.第1个H的地址。

d.*--pc的意思是吧指针递减1，并使用储存在该位置上的值。--*pc的意思是解引用pc指向的值，然后把该值减1（例如，H变成G）。

e.Ho Ho Ho!!oH oH o

注意

在两个！之间有一个空字符，但是通常该字符不会产生任何打印的效果。

f.while (*pc)检查 pc 是否指向一个空字符（即，是否指向字符串的末尾）。while 的测试条件中使用储存在指针指向位置上的值。

while (pc - str)检查pc是否与str指向相同的位置（即，字符串的开头）。while的测试条件中使用储存在指针指向位置上的值。

g.进入第1个while循环后，pc指向空字符。进入第2个while循环后，它指向空字符前面的存储区（即，str 所指向位置前面的位置）。把该字节解释成一个字符，并打印这个字符。然后指针退回到前面的字节处。永远都不会满足结束条件(pc == str)，所以这个过程会一直持续下去。

h.必须在主调程序中声明pr(): char * pr(char *);

6.字符变量占用一个字节，所以sign占1字节。但是字符常量储存为int类型，意思是'\$'通常占用2或4字节。但是实际上只使用int的1字节储存'\$'的编码。字符串"\$"使用2字节：一个字节储存'\$'的编码，一个字节储存的'\0'编码。

7.打印的内容如下:

How are ya, sweetie? How are ya, sweetie?

Beat the clock.

eat the clock.

Beat the clock.Win a toy.

Beat

chat

hat

at

t

t

at

How are ya, sweetie?

8.打印的内容如下:

faavrhee

*le*on*sm

9.下面是一种方案:

```
#include <stdio.h> // 提供fgets()和getchar()的原型
```

```
char * s_gets(char * st, int n)
```

```

{

char * ret_val;

ret_val = fgets(st, n, stdin);

if (ret_val)

{

while (*st != '\n' && *st != '\0')

st++;

if (*st == '\n')

*st = '\0';

else

while (getchar() != '\n')

continue;

}

return ret_val;

}

```

10.下面是一种方案:

```

int strlen(const char * s)

{

int ct = 0;

```

```

while (*s++)      // 或者while (*s++ != '\0')

ct++;

return(ct);

}

```

11.下面是一种方案:

```

#include <stdio.h>      // 提供 fgets()和getchar()的原型

#include <string.h>     // 提供 strchr()的原型

char * s_gets(char * st, int n)

{

char * ret_val;

char * find;

ret_val = fgets(st, n, stdin);

if(ret_val)

{

find = strchr(st, '\n');  // 查找换行符

if(find)                  // 如果地址不是 NULL,

*find = '\0';             // 在此处放置一个空字符

else

while (getchar() != '\n')

```

```

continue;

}

return ret_val;

}

```

12.下面是一种方案:

```

#include <stdio.h>    /* 提供 NULL 的定义 */

char * strblk(char * string)

{

while (*string != ' ' && *string != '\0')

string++;            /* 在第1个空白或空字符处停止 */

if(*string == '\0')

return NULL;        /* NULL 指空指针 */

else

return string;

}

```

下面是第2种方案，可以防止函数修改字符串，但是允许使用返回值改变字符串。表达式(char*)string被称为“通过强制类型转换取消const”。

```

#include <stdio.h>    /*提供 NULL 的定义*/

char * strblk(const char * string)

```

```

{

while (*string != ' ' && *string != '\0')

string++;          /*在第1个空白或空字符处停止*/

if (*string == '\0')

return NULL;      /* NULL 指空指针*/

else

return (char *)string;

}

```

13.下面是一种方案:

```

/* compare.c -- 可行方案 */

#include <stdio.h>

#include <string.h> // 提供strcmp()的原型

#include <ctype.h>

#define ANSWER "GRANT"

#define SIZE 40

char * s_gets(char * st, int n);

void ToUpper(char * str);

int main(void)

{

```

```

char try[SIZE];

puts("Who is buried in Grant's tomb?");

s_gets(try, SIZE);

ToUpper(try);

while (strcmp(try, ANSWER) != 0)

{

puts("No, that's wrong. Try again.");

s_gets(try, SIZE);

ToUpper(try);

}

puts("That's right!");

return 0;

}

void ToUpper(char * str)

{

while (*str != '\0')

{

*str = toupper(*str);

str++;

```

```

}

}

char * s_gets(char * st, int n)

{

char * ret_val;

int i = 0;

ret_val = fgets(st, n, stdin);

if (ret_val)

{

while (st[i] != '\n' && st[i] != '\0')

i++;

if (st[i] == '\n')

st[i] = '\0';

else

while (getchar() != '\n')

continue;

}

return ret_val;

}

```

A.12 第12章复习题答案

1.自动存储类别；寄存器存储类别；静态、无链接存储类别。

2.静态、无链接存储类别；静态、内部链接存储类别；静态、外部链接存储类别。

3.静态、外部链接存储类别可以被多个文件使用。静态、内部链接存储类别只能在一个文件中使用。

4.无链接。

5.关键字`extern`用于声明中，表明该变量或函数已定义在别处。

6.两者都分配了一个内含100个`int`类型值的数组。第2行代码使用`calloc()`把数组中的每个元素都设置为0。

7.默认情况下，`daisy`只对`main()`可见，以`extern`声明的`daisy`才对`petal()`、`stem()`和`root()`可见。文件2中的`extern int daisy;`声明使得`daisy`对文件2中的所有函数都可见。第1个`lily`是`main()`的局部变量。`petal()`函数中引用的`lily`是错误的，因为两个文件中都没有外部链接的`lily`。虽然文件2中有一个静态的`lily`，但是它只对文件2可见。第1个外部`rose`对`root()`函数可见，但是`stem()`中的局部`rose`覆盖了外部的`rose`。

8.下面是程序的输出：

color in main() is B

color in first() is R

color in main() is B

color in second() is G

color in main() is G

first()函数没有使用color变量，但是second()函数使用了。

9.a.声明告诉我们，程序将使用一个变量plink，该文件包含的函数都可以使用这个变量。calu_ct()函数的第1个参数是指向一个整数的指针，并假定它指向内含n个元素的数组。这里关键是要理解该程序不允许使用指针arr修改原始数组中的值。

b.不会。value和n已经是原始数据的备份，所以该函数无法更改主调函数中相应的值。这些声明的作用是防止函数修改value和n的值。例如，如果用const限定n，就不能使用n++表达式。

A.13 第13章复习题答案

1.根据文件定义，应包含#include <stdio.h>。应该把fp声明为文件指针：FILE *fp;。要给fopen()函数提供一种模式：fopen("gelatin","w")，或者"a"模式。fputs()函数的参数顺序应该反过来。输出字符串应该有一个换行符，提高可读性。fclose()函数需要一个文件指针，而不是一个文件名：fclose(fp);。下面是修改后的版本：

```
#include <stdio.h>

int main(void)
{
    FILE * fp;

    int k;

    fp = fopen("gelatin", "w");

    for (k = 0; k < 30; k++)

        fputs("Nanette eats gelatin.\n", fp);
```

```
fclose(fp);
```

```
return 0;
```

```
}
```

2.如果可以打开的话，会打开与命令行第1个参数名相同名称的文件，并在屏幕上显示文件中的每个数字字符。

```
3.a.ch = getc(fp1);
```

```
b.fprintf(fp2,"%c\n",ch);
```

```
c.putc(ch,fp2);
```

```
d.fclose(fp1); /* 关闭terky文件 */
```

注意

fp1用于输入操作，因为它识别以读模式打开的文件。与此类似，fp2以写模式打开文件，所以常用于输出操作。

4.下面是一种方案：

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main(int argc, char * argv [])
```

```
{
```

```
FILE * fp;
```

```
double n;
```

```
double sum = 0.0;
```

```

int ct = 0;

if (argc == 1)

fp = stdin;

else if (argc == 2)

{

if ((fp = fopen(argv[1], "r")) == NULL)

{

fprintf(stderr, "Can't open %s\n", argv[1]);

exit(EXIT_FAILURE);

}

}

else

{

fprintf(stderr, "Usage: %s [filename]\n", argv[0]);

exit(EXIT_FAILURE);

}

while (fscanf(fp, "%lf", &n) == 1)

{

sum += n;

```

```

++ct;

}

if(ct > 0)

printf("Average of %d values = %f\n", ct, sum / ct);

else

printf("No valid data.\n");

return 0;

}

```

5.下面是一种方案:

```

#include <stdio.h>

#include <stdlib.h>

#define BUF 256

int has_ch(char ch, const char * line);

int main(int argc, char * argv [])

{

FILE * fp;

char ch;

char line[BUF];

if(argc != 3)

```

```

{

printf("Usage: %s character filename\n", argv[0]);

exit(EXIT_FAILURE);

}

ch = argv[1][0];

if ((fp = fopen(argv[2], "r")) == NULL)

{

printf("Can't open %s\n", argv[2]);

exit(EXIT_FAILURE);

}

while (fgets(line, BUF, fp) != NULL)

{

if (has_ch(ch, line))

fputs(line, stdout);

}

fclose(fp);

return 0;

}

int has_ch(char ch, const char * line)

```

```

{

while (*line)

if(ch == *line++)

return(1);

return 0;

}

```

fgets()和 fputs()函数要一起使用，因为 fgets()会把按下 Enter 键的\n 留在字符串中， fputs()与puts()不一样，不会添加一个换行符。

6.二进制文件与文本文件的区别是，这两种文件格式对系统的依赖性不同。二进制流和文本流的区别包括是在读写流时程序执行的转换（二进制流不转换，而文本流可能要转换换行符和其他字符）。

7.a.用fprintf()储存8238201时，将其视为7个字符，保存在7字节中。用fwrite()储存时，使用该数的二进制表示，将其储存为一个4字节的整数。

b.没有区别。两个函数都将其储存为一个单字节的二进制码。

8.第1条语句是第2条语句的速记表示。第3条语句把消息写到标准错误上。通常，标准错误被定向到与标准输出相同的位置。但是标准错误不受标准输出重定向的影响。

9.可以在以"r+"模式打开的文件中读写，所以该模式最合适。"a+"只允许在文件的末尾添加内容。"w+"模式提供一个空文件，丢弃文件原来的内容。

A.14 第14章复习题答案

1.正确的关键是 struct，不是 structure。该结构模板要在左花括号前面有

一个标记，或者在右花括号后面有一个结构变量名。另外，*togs后面和模板结尾处都少一个分号。

2.输出如下：

6 1

22 Spiffo Road

S p

3.

```
struct month {
```

```
char name[10];
```

```
char abbrev[4];
```

```
int days;
```

```
int monumb;
```

```
};
```

4.

```
struct month months[12] =
```

```
{
```

```
{ "January", "jan", 31, 1 },
```

```
{ "February", "feb", 28, 2 },
```

```
{ "March", "mar", 31, 3 },
```

```

{ "April", "apr", 30, 4 },

{ "May", "may", 31, 5 },

{ "June", "jun", 30, 6 },

{ "July", "jul", 31, 7 },

{ "August", "aug", 31, 8 },

{ "September", "sep", 30, 9 },

{ "October", "oct", 31, 10 },

{ "November", "nov", 30, 11 },

{ "December", "dec", 31, 12 }

};

```

5.

```

extern struct month months [];

int days(int month)

{

int index, total;

if (month < 1 || month > 12)

return(-1); /* error signal */

else

{

```



```

for (index = 0, total = 0; index < month; index++)

total += months[index].days;

return(total);

}

}

```

注意，index比月数小1，因为数组下标从0开始。然后，用index < month代替index <= month。

6.a.要包含string.h头文件，提供strcpy()的原型：

```

typedef struct lens { /* lens 描述 */

float foclen;        /* 焦距长度，单位： mm */

float fstop;         /* 孔径 */

char brand[30];/* 品牌 */

} LENS;

LENS bigEye[10];

bigEye[2].foclen = 500;

bigEye[2].fstop = 2.0;

strcpy(bigEye[2].brand, "Remarkatar");

b.LENS bigEye[10] = { [2] = { 500, 2, "Remarkatar"} };

```

7.a.

6

Arcturan

cturan

b.使用结构名和指针:

deb.title.last

pb->title.last

c.下面是一个版本:

```
#include <stdio.h>
```

```
#include "starfolk.h"      /* 让结构定义可用 */
```

```
void prbem (const struct bem * pbem )
```

```
{
```

```
printf("%s %s is a %d-limbed %s.\n", pbem->title.first,
```

```
pbem->title.last, pbem->limbs, pbem->type);
```

```
}
```

8.a.willie.born

b.pt->born

c.scanf("%d", &willie.born);

d.scanf("%d", &pt->born);

e.scanf("%s", willie.name.lname);

```
f.scanf("%s", pt->name.lname);
```

```
g.willie.name.fname[2]
```

```
h.strlen(willie.name.fname) + strlen(willie.name.lname)
```

9.下面是一种方案:

```
struct car {
```

```
char name[20];
```

```
float hp;
```

```
float epampg;
```

```
float wbase;
```

```
int year;
```

```
};
```

10.应该这样建立函数:

```
struct gas {
```

```
float distance;
```

```
float gals;
```

```
float mpg;
```

```
};
```

```
struct gas mpgs(struct gas trip)
```

```
{
```

```

if (trip.gals > 0)

trip.mpg = trip.distance / trip.gals;

else

trip.mpg = -1.0;

return trip;

}

void set_mpgs(struct gas * ptrip)

{

if (ptrip->gals > 0)

ptrip->mpg = ptrip->distance / ptrip->gals;

else

ptrip->mpg = -1.0;

}

```

注意，第1个函数不能直接改变其主调程序中的值，所以必须用返回值才能传递信息。

```

struct gas idaho = {430.0, 14.8}; // 设置前两个成员

idaho = mpgs(idaho);             // 重置数据结构

```

但是，第2个函数可以直接访问最初的结构：

```

struct gas ohio = {583, 17.6}; //设置前两个成员

```

set_mpgs(&ohio); // 设置第3个成员

11.enum choices {no, yes, maybe};

12.char * (*pfun)(char *, char);

13.

double sum(double, double);

double diff(double, double);

double times(double, double);

double divide(double, double);

double (*pfl[4])(double, double) = {sum, diff, times, divide};

或者用更简单的形式，把代码中最后一行替换成：

typedef double (*ptype) (double, double);

ptype pfl[4] = {sum,diff, times, divide};

调用diff()函数：

pfl[1](10.0, 2.5); // 第1种表示法

(*pfl[1])(10.0, 2.5); // 等价表示法

A.15 第15章复习题答案

1.a.00000011

b.00001101

c.00111011

d.01110111

2.a.21, 025, 0x15

b.85, 0125, 0x55

c.76, 0114, 0x4C

d.157, 0235, 0x9D

3.a.252

b.2

c.7

d.7

e.5

f.3

g.28

4.a.255

b.1 (not false is true)

c.0

d.1 (true and true is true)

e.6

f.1 (true or true is true)

g.40

5.掩码的二进制是1111111；十进制是127；八进制是0177；十六进制是0x7F。

6.bitval * 2和bitval << 1都把bitval的当前值增加一倍，它们是等效的。但是mask +=bitval和mask |= bitval只有在bitval和mask没有同时打开的位时效果才相同。例如， 2 | 4得6，但是3 | 6也得6。

7.a.

```
struct tb_drives {  
  
    unsigned int diskdrives    : 2;  
  
    unsigned int                : 1;  
  
    unsigned int cdromdrives   : 2;  
  
    unsigned int                : 1;  
  
    unsigned int haddrives     : 2;  
  
};
```

b.

```
struct kb_drives {  
  
    unsigned int haddrives     : 2;  
  
    unsigned int                : 1;  
  
    unsigned int cdromdrives   : 2;  
  
    unsigned int                : 1;  
  
    unsigned int diskdrives    : 2;
```

};

A.16 第16章复习题答案

1.a.dist = 5280 * miles;有效。

b.plort = 4 * 4 + 4;有效。但是如果用户需要的是4 * (4 + 4)，则应该使用#define POD (FEET + FEET)。

c.nex = = 6;;无效（如果两个等号之间没有空格，则有效，但是没有意义）。显然，用户忘记了在编写预处理器代码时不用加=。

d.y = y + 5;有效。berg = berg + 5 * lob;有效，但是可能得不到想要的结果。est = berg + 5/y + 5;有效，但是可能得不到想要的结果。

2.#define NEW(X) ((X) + 5)

3.#define MIN(X,Y) ((X) < (Y) ? (X) : (Y))

4.#define EVEN_GT(X,Y) ((X) > (Y) && (X) % 2 == 0 ? 1 : 0)

5.#define PR(X,Y) printf("#X " is %d and " #Y " is %d\n", X,Y)

（因为该宏中没有运算符（如，乘法）作用于X和Y，所以不需要使用圆括号。）

6.a.#define QUARTERCENTURY 25

b.#define SPACE ' '

c.#define PS() putchar(' ')或#define PS() putchar(SPACE)

d.#define BIG(X) ((X) + 3)

e.#define SUMSQ(X,Y) ((X)*(X) + (Y)*(Y))

7.试试这样: `#define P(X) printf("name: "#X"; value: %d; address: %p\n", X, &X)` (如果你的实现无法识别地址专用的`%p`转换说明, 可以用`%u`或`%lu`代替。)

8.使用条件编译指令。一种方法是使用`#ifndef`:

```
#define _SKIP_ /* 如果不需要跳过代码, 则删除这条指令 */
```

```
#ifndef _SKIP_
```

```
/* 需要跳过的代码 */
```

```
#endif
```

9.

```
#ifdef PR_DATE
```

```
printf("Date = %s\n", __DATE__);
```

```
#endif
```

10.第1个版本返回`x*x`, 这只是返回了`square()`的`double`类型值。例如, `square(1.3)`会返回1.69。第2个版本返回 `(int)(x*x)`, 计算结果被截断后返回。但是, 由于该函数的返回类型是`double`, `int`类型的值将被升级为`double`类型的值, 所以1.69将先被转换成1, 然后被转换成1.00。第3个版本返回`(int)(x*x+0.5)`。加上 0.5可以让函数把结果四舍五入至与原值最接近的值, 而不是简单地截断。所以, `1.69+0.5`得2.19, 然后被截断为2, 然后被转换成2.00; 而`1.44+0.5`得1.94, 被截断为1, 然后被转换成1.00。

11.这是一种方案: `#define BOOL(X) _Generic((X), _Bool : "boolean", default : "not boolean")`12.应该把`argv`参数声明为`char *argv[]`类型。命令行参数被储存为字符串, 所以该程序应该先把`argv[1]`中的字符串转换成`double`类型的值。例如, 用`stdlib.h`库中的`atof()`函数。程序中使用了`sqrt()`函数, 所以应包含`math.h`头文件。程序在求平方根之前应排除参数为负的情况 (检查参

数是否大于或等于0) 。

13.a.qsort((void *)scores, (size_t) 1000, sizeof(double), comp);

b.下面是一个比较使用的比较函数:

```
int comp(const void * p1, const void * p2)
```

```
{
```

```
/* 要用指向int的指针来访问值 */
```

```
/* 在C中是否进行强制类型转换都可以，在C++中必须进行强制类型转换 */
```

```
const int * a1 = (const int *) p1; const int * a2 = (const int *)
```

```
p2;
```

```
if(*a1 > *a2)
```

```
return -1;
```

```
else if(*a1 == *a2)
```

```
return 0;
```

```
else
```

```
return 1;
```

```
}
```

14.a.函数调用应该类似: memcpy(data1, data2, 100 * sizeof(double));

b.函数调用应该类似: memcpy(data1, data2 + 200 , 100 * sizeof(double));

A.17 第17章复习题答案

1.定义一种数据类型包括确定如何储存数据，以及设计管理该数据的一系列函数。

2.因为每个结构包含下一个结构的地址，但是不包含上一个结构的地址，所以这个链表只能沿着一个方向遍历。可以修改结构，在结构中包含两个指针，一个指向上一个结构，一个指向下一个结构。当然，程序也要添加代码，在每次新增结构时为这些指针赋正确的地址。

3.ADT是抽象数据类型，是对一种类型属性集和可以对该类型进行的操作的正式定义。ADT应该用一般语言表示，而不是用某种特殊的计算机语言，而且不应该包含实现细节。

4.直接传递变量的优点：该函数查看一个队列，但是不改变其中的内容。直接传递队列变量，意味着该函数使用的是原始队列的副本，这保证了该函数不会更改原始的数据。直接传递变量时，不需要使用地址运算符或指针。

直接传递变量的缺点：程序必须分配足够的空间储存整个变量，然后拷贝原始数据的信息。如果变量是一个大型结构，用这种方法将花费大量的时间和内存空间。

传递变量地址的优点：如果待传递的变量是大型结构，那么传递变量的地址和访问原始数据会更快，所需的内存空间更少。

传递变量地址的缺点：必须记得使用地址运算符或指针。在K&R C中，函数可能会不小心改变原

始数据，但是用ANSI C中的`const`限定符可以解决这个问题。

5.a.

类型名： 栈

类型属性: 可以储存有序项

类型操作: 初始化栈为空

确定栈是否为空

确定栈是否已满

从栈顶添加项（压入项）

从栈顶删除项（弹出项）

b.下面以数组形式实现栈，但是这些信息只影响结构定义和函数定义的细节，不会影响函数原型的接口。

```
/* stack.h — 栈的接口 */
```

```
#include <stdbool.h>
```

```
/* 在这里插入 Item 类型 */
```

```
/* 例如: typedef int Item; */
```

```
#define MAXSTACK 100
```

```
typedef struct stack
```

$$\{$$

```
Item items[MAXSTACK]; /* 儲存信息 */
```

```
int top;           /* 第1个空位的索引 */
```

```

} Stack;

```

```
/*操作: 初始化栈 */
```

```

/* 前提条件:      ps 指向一个栈                                */
/* 后置条件:      该栈被初始化为空                                */

void InitializeStack(Stack * ps);

/* 操作:          检查栈是否已满                                */

/* 前提条件:      ps 指向之前已被初始化的栈                                */

/* 后置条件:      如果栈已满, 该函数返回true; 否则, 返回false
*/

bool FullStack(const Stack * ps);

/* 操作:          检查栈是否为空                                */

/* 前提条件:      ps 指向之前已被初始化的栈                                */

/* 后置条件:      如果栈为空, 该函数返回true; 否则, 返回false
*/

bool EmptyStack(const Stack *ps);

/* 操作:          把项压入栈顶                                */

/* 前提条件:      ps 指向之前已被初始化的栈                                */

/*          item 是待压入栈顶的项                                */

/* 后置条件:      如果栈不满, 把 item 放在栈顶, 该函数返回ture;
*/

/*          否则, 栈不变, 该函数返回 false                                */

bool Push(Item item, Stack * ps);

```

```

/* 操作：          从栈顶删除项                                */

/* 前提条件：      ps 指向之前已被初始化的栈                    */

/* 后置条件：      如果栈不为空，把栈顶的item拷贝到*pitem,
*/

/*          删除栈顶的item，该函数返回ture；                    */

/*          如果该操作后栈中没有项，则重置该栈为空。
*/

/*          如果删除操作之前栈为空，栈不变，该函数返回false
*/

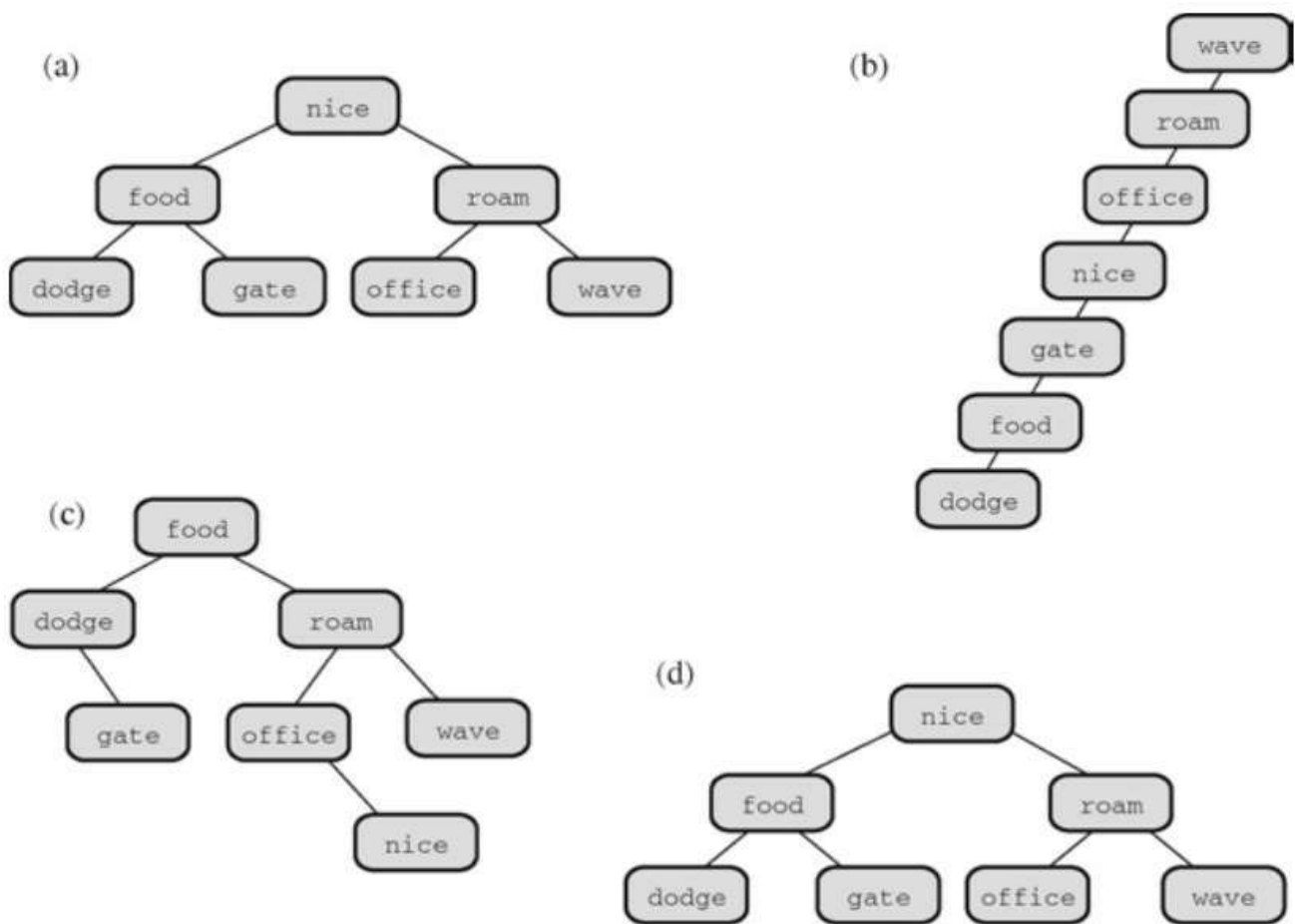
bool Pop(Item *pitem, Stack * ps);

```

6.比较所需的最大次数如下：

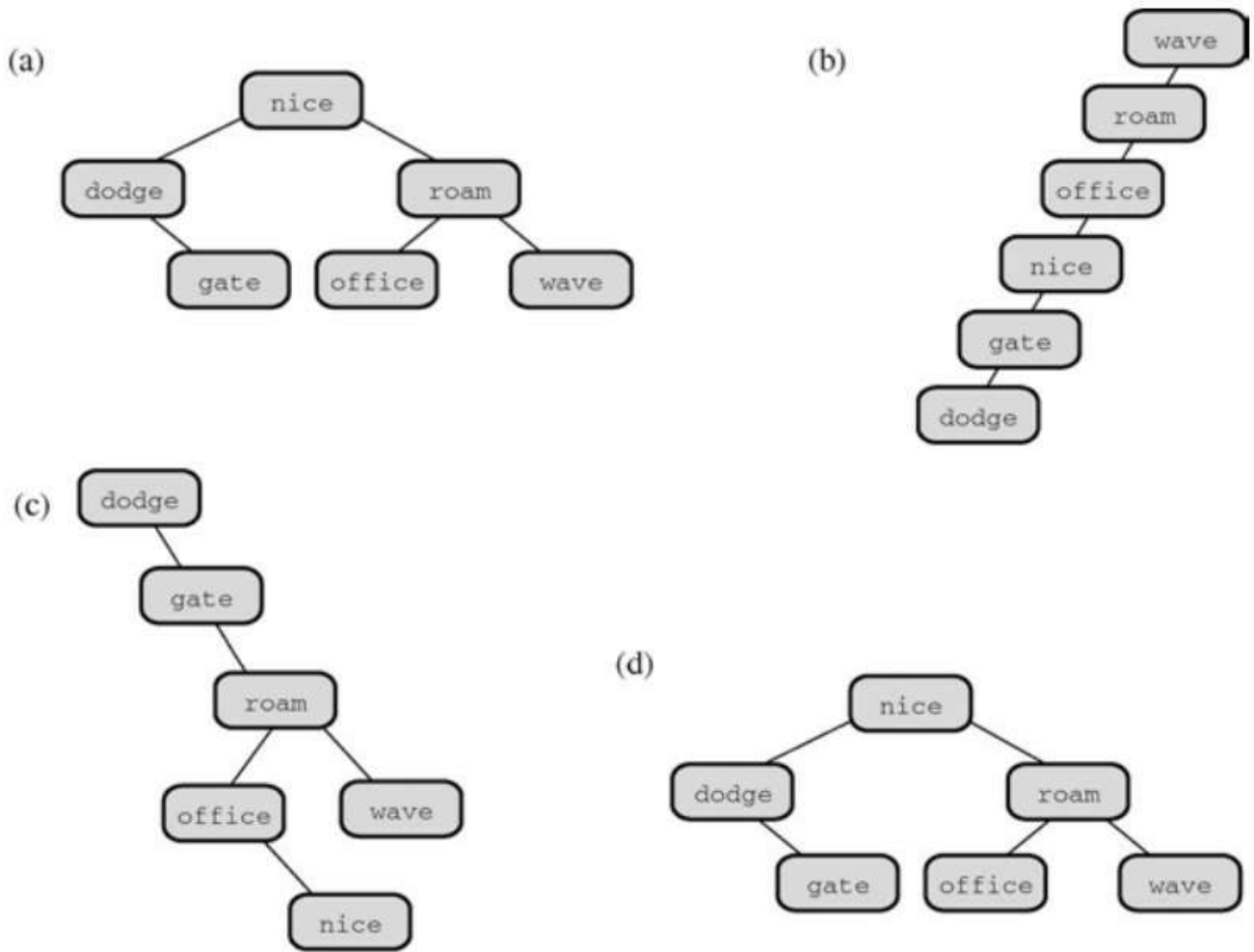
项	顺序查找	二分查找
3	3	2
1023	1023	10
65535	65535	16

7.见图A.1。



图A.1 单词的二分查找树

8.见图A.2。



图A.2 删除项后的单词二分查找树

[1].这句英文翻译成中文是“这句话是出色的捷克人”。显然不知所云，这就是语言中的语义错误。——译者注

[2].thrice_n本应表示n的3倍，但是 $3 + n$ 表示的并不是n的3倍，应该用 $3*n$ 来表示。——译者注